Module 1: Deep Learning Fundamentals

Cognitive Computing using TensorFlow (PEC-CS7015) | BE Computer Engineering, Sem VII | 8 hrs | Bloom's L1, L2 | CO1 (about 18% of the exam)

Module 1 is the foundation of the whole course. Every formula is written as a proper equation in a blue box, followed by a plain-English reading and the meaning of each symbol. For every long answer use this flow: **Definition → Diagram → Formula → Example → Advantages → Disadvantages → Applications.**

How to read the equations
Σ means 'add up'; **η** (eta) = learning rate; **θ** (theta) = threshold or the parameters; **σ** (sigma) = the sigmoid function; **ŷ** (y-hat) = predicted output; **y** or **t** = true target; **f′** = derivative of f; **∂L/∂w** = how much the loss L changes when w changes.

Topic map

1. History of Deep Learning
2. Tensors and Computation Graphs
3. McCulloch-Pitts Neuron and Thresholding Logic
4. Perceptron and Perceptron Learning Algorithm
5. Limitations of Single-Layer Perceptron (XOR)
6. Multilayer Perceptrons (MLPs) and Representation Power
7. Sigmoid Neurons
8. Gradient Descent, Saturation and Error Correction in MLP
9. Activation Functions (Sigmoid, Tanh, ReLU)
10. Loss Functions (MSE, Cross-Entropy)
11. Vanishing Gradient Problem
12. Basic Variants of Gradient Descent
13. Quick revision sheet and likely questions

1. History of Deep Learning

Definition: Deep Learning (DL) is a subset of Machine Learning that uses artificial neural networks with **many hidden layers** to automatically learn hierarchical representations of data (raw input → low-level features → high-level concepts). **Hierarchy:** AI ⊃ Machine Learning ⊃ Deep Learning.

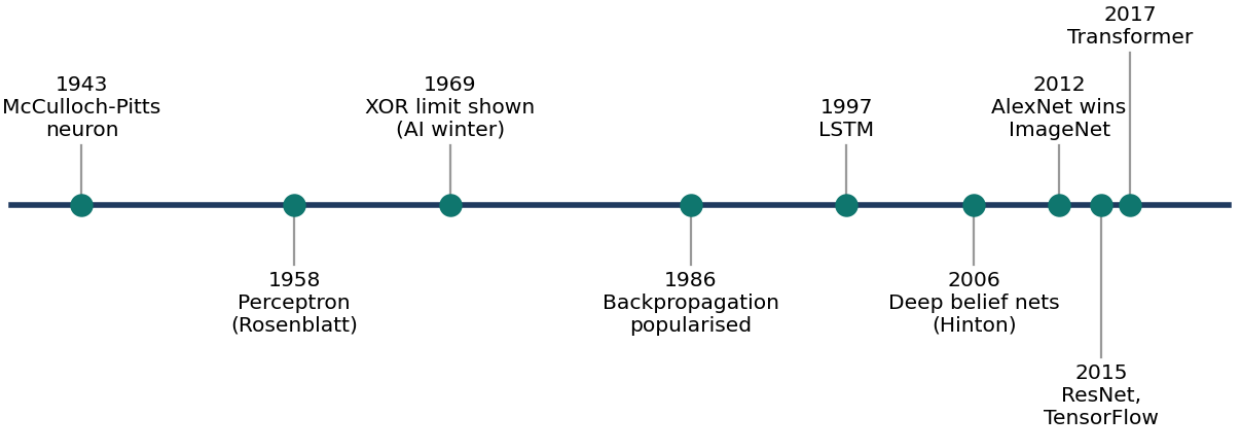


Figure 1: Key milestones in the history of neural networks and deep learning.

Year	Milestone	Why it matters
1943	McCulloch and Pitts: first mathematical neuron	Birth of neural computation
1949	Hebb's learning rule: 'neurons that fire together wire together'	First idea of learning by changing weights
1958	Rosenblatt invents the Perceptron	First trainable neural model
1960	Widrow and Hoff: ADALINE, delta (LMS) rule	Gradient-based learning
1969	Minsky and Papert show a single perceptron cannot solve XOR	Funding collapses: first **AI winter**
1974-86	Werbos (1974); Rumelhart, Hinton and Williams (1986) popularise **backpropagation**	Multilayer networks become trainable
1989	LeCun uses a CNN with backprop for handwritten zip-code digits	Birth of practical CNNs
1997	Hochreiter and Schmidhuber: LSTM	Solves vanishing gradient in sequences
1990s	SVMs and kernel methods dominate; neural nets fall out of favour	Limited data and compute (second AI winter)
2006	Hinton: Deep Belief Networks, greedy layer-wise pre-training	The term 'deep learning' revives
2009	ImageNet dataset; GPUs start to be used for training	Data and compute arrive
2012	**AlexNet** wins ImageNet by a huge margin	The DL boom begins
2014	GANs (Goodfellow), seq2seq, Adam optimizer	Generative and sequence models
2015	ResNet (152 layers); TensorFlow released	Very deep nets trainable; easy tooling
2017	Transformer: 'Attention Is All You Need'	Basis of modern language models
2018+	BERT, GPT family, large language models	Foundation models

Why did deep learning succeed only after 2010? (important)

1. **Big data:** ImageNet, web-scale text and logs.
2. **Compute:** GPUs and TPUs make matrix multiplication massively parallel.
3. **Algorithmic advances:** ReLU, dropout, batch normalization, better initialization, Adam.
4. **Frameworks:** TensorFlow, PyTorch and Keras make building models easy.

Applications: image recognition, voice assistants (Alexa, Siri), machine translation, medical imaging, self-driving cars, recommendation systems, chatbots.

2. Tensors and Computation Graphs

2.1 Tensor

Definition: A tensor is an **n-dimensional array of numbers**. It is the basic data structure of deep learning (hence the name TensorFlow).

Rank	Name	Example	Shape	Real use
0	Scalar	7	( )	A loss value, a learning rate
1	Vector	[1, 2, 3]	(3,)	Features of one sample, bias vector
2	Matrix	2×3 grid of numbers	(2, 3)	Weight matrix, a table of data
3	3-D tensor	Stack of matrices	(batch, time, features)	Time series, text
4	4-D tensor	Batch of images	(batch, height, width, channels)	CNN input, e.g. (32, 28, 28, 1)

Key properties: rank (number of dimensions), **shape** (size along each dimension), **dtype** (float32, int32, ...).

```
import tensorflow as tf
scalar = tf.constant(5)                    # rank 0
vector = tf.constant([1.0, 2.0, 3.0])      # rank 1, shape (3,)
matrix = tf.constant([[1, 2], [3, 4]])     # rank 2, shape (2, 2)
print(matrix.shape, matrix.dtype)
```

Why tensors? Any data (image, text, audio) can be stored in the same way, and tensor operations (matrix multiplication) run very fast on GPUs.

2.2 Computation graph

Definition: A computation graph is a **directed acyclic graph (DAG)** in which **nodes** are operations or variables and **edges** carry tensors between them.

Example: the function below, with a = 2, b = 3, c = 4:

EXAMPLE FUNCTION

$$y = (a + b) \times c$$

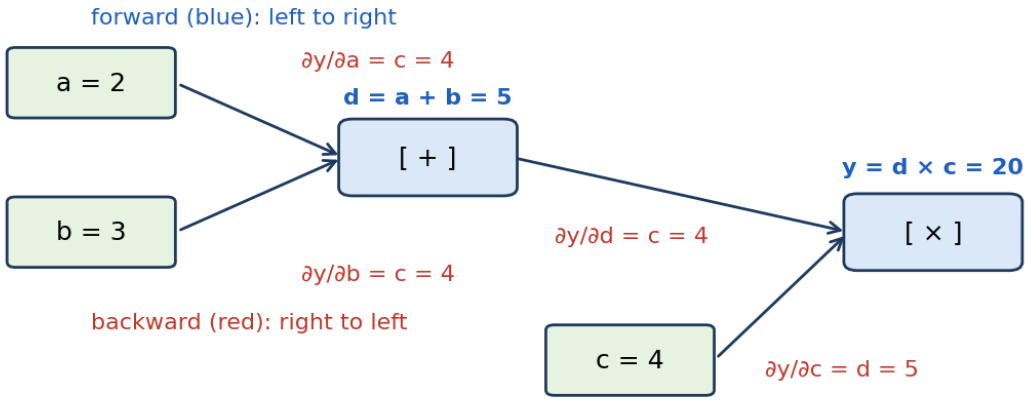


Figure 2: Computation graph of y = (a + b) × c with forward values (blue) and backward gradients (red).

Backward pass uses the chain rule on each node:

GRADIENTS BY THE CHAIN RULE

$$\frac{\partial y}{\partial a} = \frac{\partial y}{\partial d} \cdot \frac{\partial d}{\partial a} = c \cdot 1 = 4 \qquad \frac{\partial y}{\partial b} = c = 4 \qquad \frac{\partial y}{\partial c} = d = 5$$

- **Forward pass:** values flow left to right and give the output.
- **Backward pass (backpropagation):** gradients flow right to left using the chain rule; this is called **automatic differentiation**.
- The graph also enables optimization, parallel execution and deployment on different devices.

Feature	Static graph (TF 1.x)	Dynamic graph (TF 2.x eager, PyTorch)
Built	Defined first, then run in a session	Built on the fly as the code runs
Debugging	Harder	Easy (normal Python debugging)
Optimization	Better whole-graph optimization	Less, but tf.function can compile it
Flexibility	Low	High

3. McCulloch-Pitts (MP) Neuron and Thresholding Logic

Definition: The MP neuron (1943) is the earliest mathematical model of a biological neuron. It takes **binary inputs**, adds them up, and **fires (outputs 1)** if the sum reaches a **threshold θ**.

MP NEURON

$$g(x) = \sum_{i=1}^{n} x_i$$

$$y = 1 \ \text{ if } \ g(x) \geq \theta \qquad\qquad y = 0 \ \text{ otherwise}$$

where: $\mathbf{x_i}$ = binary inputs (0 or 1) | **g(x)** = the sum of the inputs | **θ** = threshold, fixed by hand | **y** = binary output

Some inputs may be **inhibitory**: if any inhibitory input is 1, the output is forced to 0.

Thresholding logic: logic gates with an MP neuron

Function	Threshold θ	Explanation
AND (n inputs)	θ = n	Fires only if all inputs are 1
OR	θ = 1	Fires if at least one input is 1
NOR	θ = 0, all inputs inhibitory	Fires only when all inputs are 0
NOT	θ = 0, inhibitory input	Inverts the input

Example: 2-input AND with θ = 2. For (1,1) the sum is 2, which is ≥ 2, so y = 1. For (1,0) the sum is 1, which is less than 2, so y = 0.

Geometric view: for two inputs the line $x_1 + x_2 = \theta$ divides the plane in two regions, so an MP neuron is a **linear classifier**.

Limitations: inputs and outputs are only binary; all inputs are equally important (no weights); the threshold is set manually (**no learning**); cannot represent XOR.

4. Perceptron and the Perceptron Learning Algorithm

Definition: The perceptron (Rosenblatt, 1958) is an improved MP neuron with **real-valued inputs, adjustable weights and a bias**, and it can **learn** its weights from data.

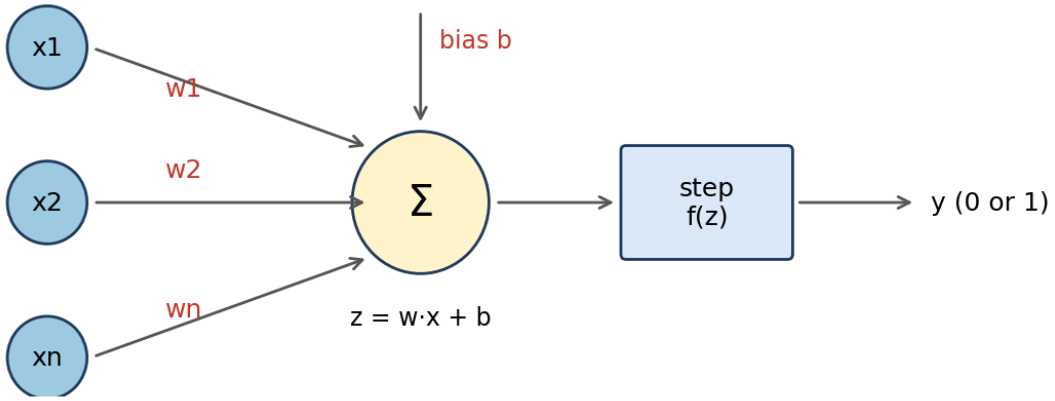


Figure 3: A perceptron: weighted sum of the inputs plus a bias, followed by a step function.

PERCEPTRON

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w} \cdot \mathbf{x} + b$$

$$y = 1 \text{ if } z \geq 0 \qquad y = 0 \text{ otherwise}$$

where: **w** = weights (learned) | **b** = bias = minus the threshold | **z** = weighted sum | **step** = output is 1 when z is 0 or more

Point	MP neuron	Perceptron
Inputs	Binary only	Real-valued
Weights	None (all equal)	Learnable
Threshold	Fixed by hand	Learned through the bias
Learning	No	Yes (perceptron rule)
Year	1943	1958

Perceptron Learning Algorithm (PLA)

1. Initialise the weights w and bias b to small random values (or zeros).
2. For each training example (x, t), compute the output y = step(w·x + b). Here t is the target (0 or 1).
3. Update the weights with the rule below.
4. Repeat over the dataset (epochs) until there are no mistakes or a maximum number of epochs is reached.

PERCEPTRON UPDATE RULE

$$w \leftarrow w + \eta\,(t - y)\,x$$

$$b \leftarrow b + \eta\,(t - y)$$

where: **η** = learning rate ($0 < \eta \leq 1$) | **t** = target output | **y** = actual output | **t − y** = the error: +1, 0 or −1

- If y = t: (t − y) = 0, so **no change**.
- If t = 1 but y = 0 (output too low): the weights move **towards** x, so the output rises next time.

- If t = 0 but y = 1 (output too high): the weights move **away from** x.

Worked example: learning the AND gate (η = 1, start w = (0, 0), b = 0)

Input (x_1, x_2)	Target t	z	Output y	Update after this sample (epoch 1)
(0, 0)	0	0	1	w = (0, 0), b = -1
(0, 1)	0	-1	0	no change
(1, 0)	0	-1	0	no change
(1, 1)	1	-1	0	w = (1, 1), b = 0

After **6** passes (epochs) over the data there are no mistakes left, with final weights **w = (2, 1)** and **b = -3**. Check: $z = 2x_1 + 1x_2 - 3$ is ≥ 0 only for (1, 1). That is exactly AND.

Convergence theorem
If the data is **linearly separable**, the perceptron algorithm is guaranteed to converge in a finite number of steps. If it is not, the algorithm never converges.

Advantages: simple and fast; guaranteed convergence on separable data; works online (one sample at a time).

Disadvantages: only linear problems; only binary output; the step function is not differentiable, so gradient descent cannot be used; sensitive to noise.

5. Limitations of the Single-Layer Perceptron (SLP)

1. **Cannot solve non-linearly separable problems such as XOR** (Minsky and Papert, 1969).
2. Output is binary (hard decision), with no probability.
3. The step function has zero gradient almost everywhere, so gradient-based training is impossible.
4. Does not converge on noisy or overlapping data.
5. Cannot learn complex hierarchical features.

The XOR problem

x_1	x_2	XOR output
0	0	0
0	1	1
1	0	1
1	1	0

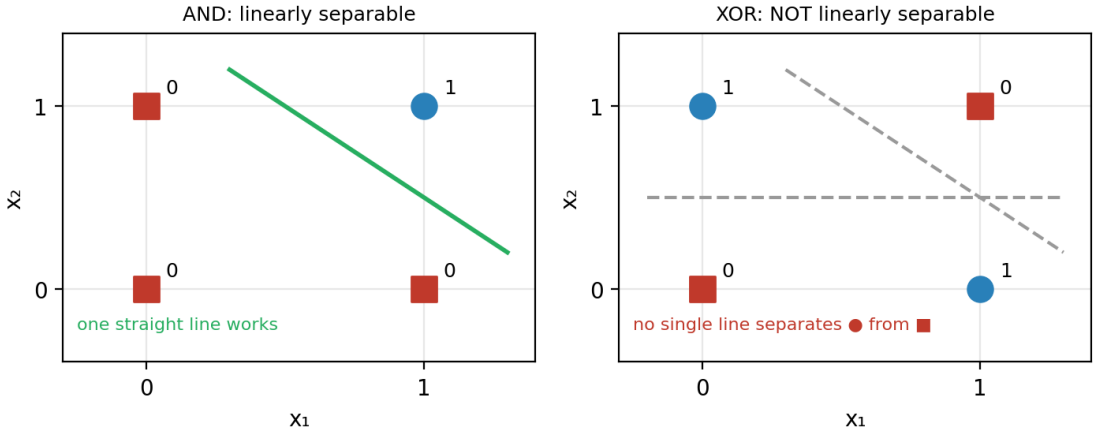


Figure 4: AND can be separated by one straight line; XOR cannot.

Algebraic proof. A perceptron needs weights and bias satisfying all four conditions at once:

$$b<0 \quad (0,0) \qquad w_1+b\geq 0 \quad (1,0) \qquad w_2+b\geq 0 \quad (0,1) \qquad w_1+w_2+b<0 \quad (1,1)$$

Adding the middle two gives $w_1 + w_2 + 2b \geq 0$, so $w_1 + w_2 + b \geq -b$, which is positive because $b < 0$. This contradicts the last condition. **So no such weights exist.**

Solution: add a **hidden layer** (an MLP). XOR can be written as:

$$x_1 \oplus x_2 = (x_1 \text{ OR } x_2) \text{ AND NOT}(x_1 \text{ AND } x_2)$$

The hidden layer computes OR and NAND; the output neuron ANDs them.

6. Multilayer Perceptrons (MLPs) and Representation Power

Definition: An MLP is a feedforward network with an **input layer, one or more hidden layers and an output layer**. Neurons in adjacent layers are fully connected and hidden neurons use a **non-linear activation**.

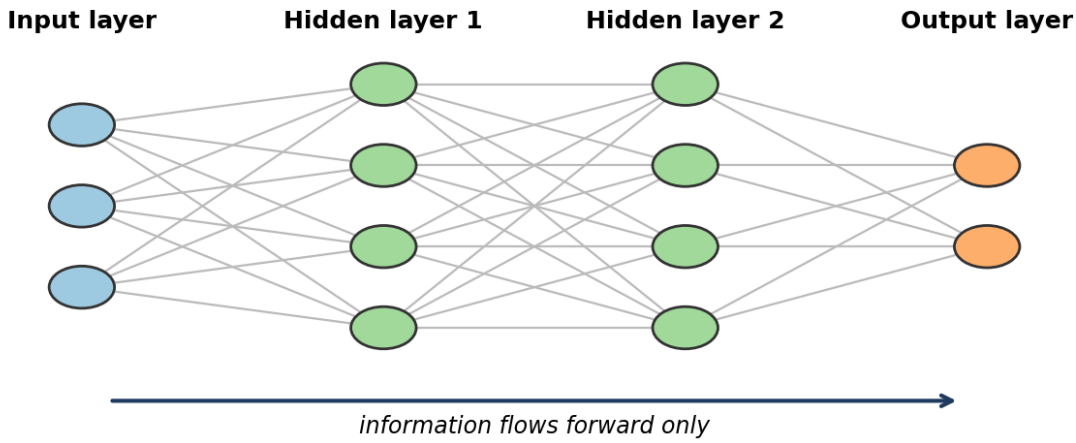


Figure 5: An MLP with two hidden layers (3-4-4-2).

FORWARD PROPAGATION IN LAYER L

$$a^{(l)} = W^{(l)}\, h^{(l-1)} + b^{(l)}$$

$$h^{(l)} = f(a^{(l)})$$

Why non-linearity matters: if all activations were linear, a stack of layers would collapse into a single linear layer, so depth would be useless:

$$W_2(W_1\, x) = (W_2 W_1)x$$

Representation power of MLPs

- **Universal Approximation Theorem** (Cybenko 1989, Hornik 1991): a feedforward network with **one hidden layer** and enough neurons with a non-linear activation (e.g. sigmoid) can approximate **any continuous function** on a bounded region to any accuracy.
- **Boolean functions:** any Boolean function of n inputs can be built with one hidden layer (possibly needing up to 2^n hidden neurons).

- **Intuition:** each hidden neuron carves out a half-space; combining many of them builds 'bumps' and arbitrary regions.
- **Why go deep then?** The theorem says a network *exists*, not that it is *learnable* or *efficient*. A shallow net may need exponentially many neurons, whereas a deep net reuses features (edges → shapes → objects) with far fewer parameters.

UNIVERSAL APPROXIMATION (ONE HIDDEN LAYER)

$$F(x) = \sum_{j=1}^{M} \alpha_j \, \sigma(w_j \cdot x + b_j) \qquad \Rightarrow \qquad |F(x) - f(x)| < \varepsilon$$

where: **F(x)** = network output | **M** = number of hidden neurons | **f(x)** = the target function | **ε** = any small error we want

Feature	Single-layer perceptron	MLP
Layers	Input and output only	At least one hidden layer
Decision boundary	Linear	Non-linear
XOR	Cannot solve	Can solve
Activation	Step	Sigmoid, tanh, ReLU, ...
Training	Perceptron rule	Backpropagation with gradient descent
Power	Very limited	Universal approximator

Applications: classification (spam, fraud), regression (house price), pattern recognition, tabular data.

Advantages: learns complex non-linear patterns; universal approximation; handles classification and regression.

Disadvantages: many parameters (overfitting risk); black box; needs lots of data and compute; can get stuck in local minima; sensitive to hyperparameters.

7. Sigmoid Neurons

Why needed: the perceptron's step function is abrupt: a tiny weight change can flip the output from 0 to 1, and it is not differentiable. A **sigmoid neuron** replaces it with a smooth S-shaped curve.

SIGMOID NEURON

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

where: **σ(z)** = output between 0 and 1 (can be read as a probability) | **σ′(z)** = derivative, maximum value 0.25 at z = 0

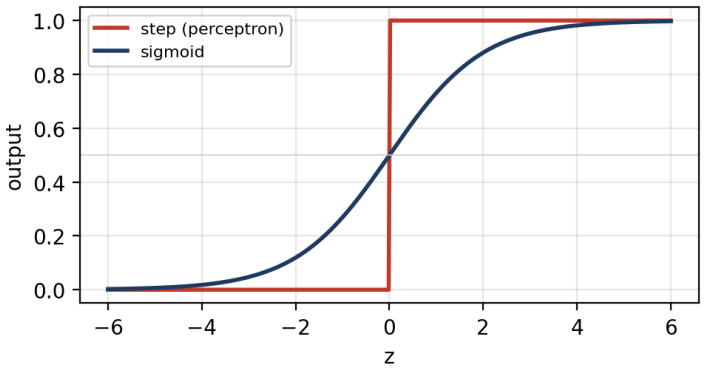


Figure 6: The step function jumps; the sigmoid changes smoothly.

Feature	Perceptron (step)	Sigmoid neuron
Output	0 or 1	Continuous, in (0, 1)
Differentiable	No	Yes
Small weight change	Can flip the output suddenly	Small change in output
Probabilistic meaning	No	Yes
Gradient descent	Not possible	Possible

Advantages: smooth and differentiable; probabilistic output; good for the output layer of binary classification.

Disadvantages: saturates at both ends (gradient close to 0, which leads to vanishing gradients); output is not zero-centred (slows convergence); the exponential is costly to compute.

8. Gradient Descent, Saturation and Error Correction in MLP

8.1 Gradient Descent (GD)

Definition: GD is an iterative optimization algorithm that minimises a loss function J(θ) by repeatedly moving the parameters in the direction of the **negative gradient** (steepest descent).

GRADIENT DESCENT UPDATE RULE

$$\theta \leftarrow \theta - \eta \frac{\partial J}{\partial \theta}$$

where: **θ** = a weight or bias | **η** = learning rate (step size) | **∂J/∂θ** = slope of the loss with respect to θ

Mountain analogy: you are blindfolded on a hill and want to reach the valley. Feel the slope under your feet and step downhill each time. The step size is the learning rate.

Example: for J(w) = w² starting at w = 4 with η = 0.1, each step multiplies w by 0.8, so w goes 4 → 3.2 → 2.56 → 2.05 → ... → 0 (the minimum).

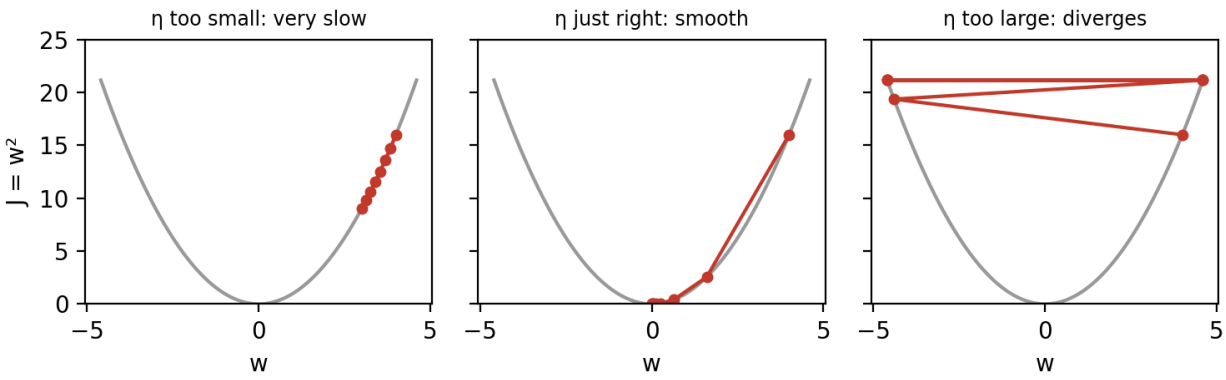


Figure 7: Effect of the learning rate on $J = w^2$ (starting at $w = 4$).

Learning rate η	Behaviour
Too small	Very slow convergence
Just right	Smooth, fast convergence
Too large	Overshoots, oscillates, may diverge

8.2 Error correction: how an MLP is trained

1. **Forward pass:** compute the prediction.
2. **Compute the loss:** compare with the target (the error).
3. **Backward pass (backpropagation):** use the chain rule to compute ∂L/∂w for every weight, from output back to input.
4. **Update the weights** with gradient descent.
5. Repeat for many epochs.

For an output sigmoid neuron with squared error, the error signal that is sent backwards is:

$$L = \tfrac{1}{2}(y - \hat{y})^2 \quad \Rightarrow \quad \delta = (\hat{y} - y)\hat{y}(1 - \hat{y}) \quad \Rightarrow \quad \frac{\partial L}{\partial w} = \delta \times (\text{input to that weight})$$

The full backpropagation derivation is in Module 2.

8.3 Saturation in an MLP

Definition: a neuron is **saturated** when its input z is very large (positive or negative), so a sigmoid or tanh output sits on the flat ends of the curve (near 0 or 1) where the **derivative is almost zero**.

- **Consequence:** gradient ≈ 0 means the weight update ≈ 0, so learning stalls for that neuron.
- It happens with large weights, badly scaled inputs or poor initialization.
- It slows or blocks the training of the whole network.

Remedies: normalise the inputs; initialise small weights (Xavier / He); use ReLU; use batch normalization; lower the learning rate.

9. Activation Functions

Definition: An activation function is a (usually non-linear) function applied to a neuron's weighted sum z that decides the neuron's output. It adds **non-linearity**, so the network can learn complex mappings. **Good properties:** non-linear, differentiable, cheap to compute, avoids saturation, preferably zero-centred.

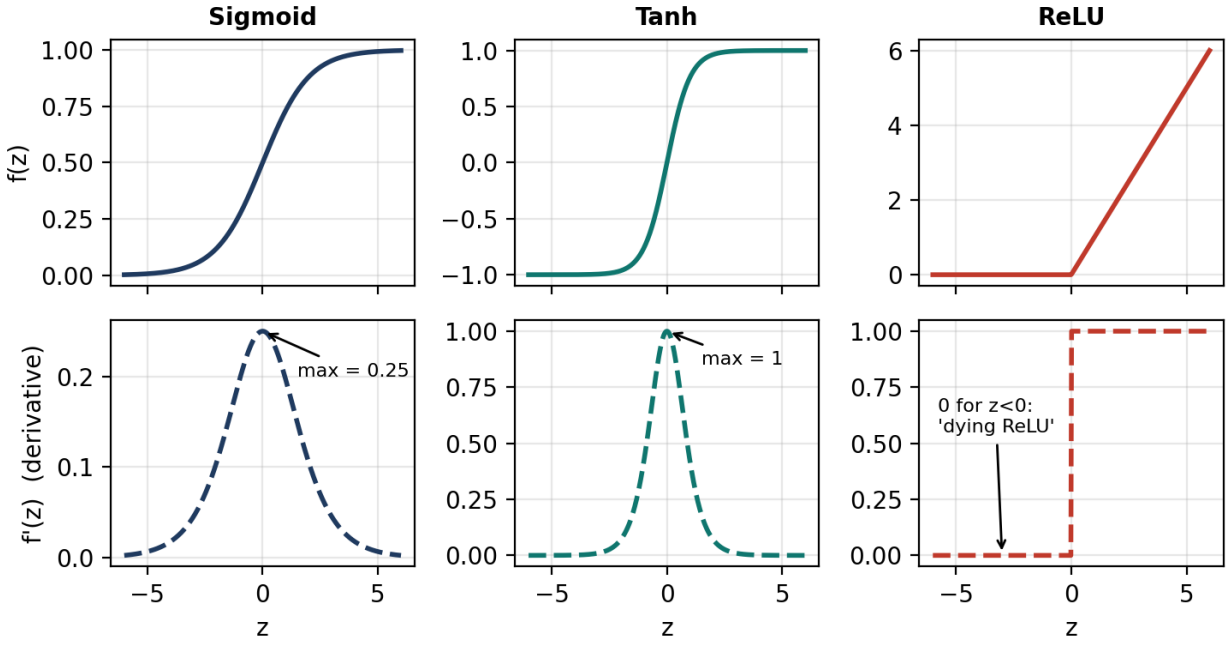


Figure 8: Sigmoid, tanh and ReLU (top row) and their derivatives (bottom row).

9.1 Tanh (hyperbolic tangent)

TANH

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} = 2\,\sigma(2z) - 1$$

$$\tanh'(z) = 1 - \tanh^{2}(z)$$

Range: (−1, 1), **zero-centred**. Maximum derivative = 1 (at z = 0).

Advantages: zero-centred (balanced positive and negative outputs, so faster convergence than sigmoid); stronger gradients than sigmoid (max 1 vs 0.25).

Disadvantages: still saturates for large |z| (vanishing gradient); costly exponentials. **Used in:** hidden layers of RNN/LSTM and older MLPs.

9.2 ReLU (Rectified Linear Unit)

RELU

$$f(z) = \max(0,\, z)$$

$$f'(z) = 1 \;\text{ if }\; z > 0 \qquad\qquad f'(z) = 0 \;\text{ if }\; z < 0$$

Range: [0, ∞).

Advantages: very cheap to compute; **does not saturate for z > 0** (reduces vanishing gradient); sparse activation (many neurons output 0); converges much faster (about 6 times faster than tanh

in AlexNet).

Disadvantages: the **dying ReLU problem**: a neuron stuck with z < 0 always outputs 0, has zero gradient and never recovers; not zero-centred; unbounded output. (Fixes: Leaky ReLU, Parametric ReLU, ELU; see Module 4.) **Used in:** hidden layers of almost all CNNs and deep MLPs (the default choice).

9.3 Softmax (for multi-class output)

SOFTMAX

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

It turns K scores into K probabilities that add up to 1.

Comparison table (high-scoring answer)

Feature	Sigmoid	Tanh	ReLU
Formula	$1/(1+e^{-z})$	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	max(0, z)
Range	(0, 1)	(−1, 1)	[0, ∞)
Zero-centred	No	Yes	No
Max derivative	0.25	1	1
Vanishing gradient	Severe	Yes (less)	No (for z > 0)
Computation	Costly	Costly	Very cheap
Main problem	Saturation	Saturation	Dying neurons
Typical use	Binary output layer	RNN/LSTM hidden	Hidden layers of CNN/MLP

10. Loss Functions

Definition: A loss (cost) function measures **how far the prediction is from the true value**. Training means minimising it. *Loss* is for one sample; *cost* is the average over the dataset.

10.1 Mean Squared Error (MSE)

MEAN SQUARED ERROR (REGRESSION)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Example: true values [3, 5], predictions [2, 7]. Errors are 1 and −2, so:

$$\text{MSE} = \frac{1^2 + (-2)^2}{2} = \frac{1+4}{2} = 2.5$$

Advantages: simple; smooth gradient; mathematically convenient. **Disadvantages: sensitive to outliers**; with a sigmoid output it learns slowly when the neuron saturates; not ideal for classification.

10.2 Cross-Entropy loss

BINARY CROSS-ENTROPY (2 CLASSES)

$$L_{\text{BCE}} = -\left[t \log \hat{y} + (1-t) \log (1-\hat{y})\right]$$

CATEGORICAL CROSS-ENTROPY (K CLASSES, SOFTMAX OUTPUT)

$$L_{\text{CCE}} = -\sum_{k=1}^{K} t_k \log \hat{y}_k$$

Example (BCE): true t = 1.

$$\hat{y} = 0.9 \;\Rightarrow\; L = -\log(0.9) = 0.105 \qquad (\text{low loss: good prediction})$$

$$\hat{y} = 0.1 \;\Rightarrow\; L = -\log(0.1) = 2.30 \qquad (\text{high loss: bad prediction})$$

So **confident wrong predictions are punished heavily**.

Why cross-entropy beats MSE for classification with a sigmoid / softmax output: compare the gradient with respect to z.

GRADIENT COMPARISON

$$\frac{\partial L_{\text{MSE}}}{\partial z} = (\hat{y} - t)\,\hat{y}\,(1-\hat{y}) \qquad \frac{\partial L_{\text{CE}}}{\partial z} = \hat{y} - t$$

The MSE gradient contains the factor ŷ(1 − ŷ), which becomes tiny when the neuron saturates, so learning stalls. The cross-entropy gradient does not, so learning stays fast. Cross-entropy also comes from maximum likelihood estimation.

Advantages: fast convergence; probabilistic meaning; strong penalty for confident mistakes.
Disadvantages: undefined at log(0) (needs clipping); sensitive to class imbalance and noisy labels.

Feature	MSE	Cross-entropy
Task	Regression	Classification
Output layer	Linear	Sigmoid / Softmax
Measures	Squared distance	Difference between probability distributions
Gradient w.r.t. z (sigmoid)	(ŷ−t) ŷ(1−ŷ): can vanish	(ŷ−t): stays strong
Outlier sensitivity	High	Less critical
Convergence in classification	Slower	Faster

TensorFlow: `loss='mse'`, `'binary_crossentropy'`, `'categorical_crossentropy'` (or `'sparse_categorical_crossentropy'` when labels are integers).

11. Vanishing Gradient Problem

Definition: During backpropagation in a deep network, gradients become **exponentially smaller** as they travel from the output layer back towards the early layers, so the early layers learn extremely slowly or not at all.

Cause (chain rule): the gradient for an early weight is a **product** of many terms:

GRADIENT OF AN EARLY WEIGHT IN AN N-LAYER NETWORK

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}}\, f'(z_n)\, w_n \;\; f'(z_{n-1})\, w_{n-1} \cdots f'(z_1)\, x$$

For a sigmoid, every factor f′(z) is at most 0.25. Multiplying n such numbers gives about 0.25^n:

$$n = 10 \quad \Rightarrow \quad 0.25^{10} \approx 10^{-6} \quad \text{(practically zero)}$$

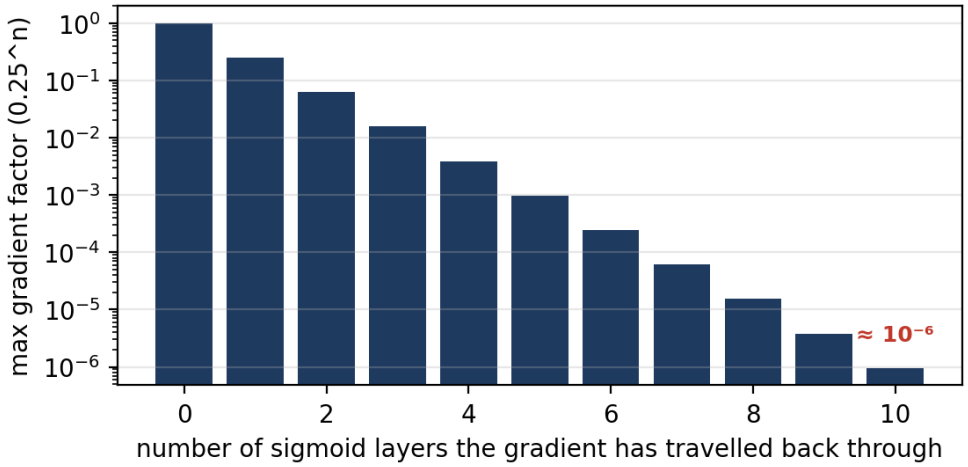


Figure 9: The largest possible gradient factor shrinks exponentially with depth (log scale).

Effects: early layers do not learn useful features; training is slow and stalls; accuracy stays poor; worse with sigmoid/tanh and poor initialization.

Solutions (important list)

1. Use **ReLU** and variants (Leaky ReLU, ELU).
2. Proper **weight initialization** (Xavier for tanh/sigmoid, He for ReLU).
3. **Batch normalization.**
4. **Residual / skip connections** (ResNet).
5. **LSTM / GRU** gates for sequence models.
6. Greedy layer-wise pre-training (historical solution).

Exploding gradient (the opposite problem): gradients grow exponentially (when weights are large), giving unstable training and NaN values. Fixed by **gradient clipping**, careful initialization and a smaller learning rate.

Vanishing gradient	Exploding gradient
Gradients shrink towards 0	Gradients grow very large
Early layers stop learning	Weights become huge, loss becomes NaN
Caused by sigmoid/tanh and small weights	Caused by large weights, deep or recurrent nets
Fix: ReLU, batch norm, residuals, LSTM	Fix: gradient clipping, good initialization

12. Basic Variants of Gradient Descent (Conceptual)

The three variants differ in **how much data is used to compute one gradient update**.

UPDATE RULES

$$\text{Batch}: \quad \theta \leftarrow \theta - \eta \frac{1}{N} \sum_{i=1}^{N} \nabla L_i(\theta)$$

$$\text{Stochastic (SGD)}: \quad \theta \leftarrow \theta - \eta \nabla L_i(\theta)$$

$$\text{Mini-batch}: \quad \theta \leftarrow \theta - \eta \frac{1}{b} \sum_{i \in B} \nabla L_i(\theta)$$

where: **N** = total number of samples | **b** = mini-batch size (e.g. 32, 64, 128) | **B** = the randomly chosen mini-batch | $\mathbf{L_i}$ = loss on sample i

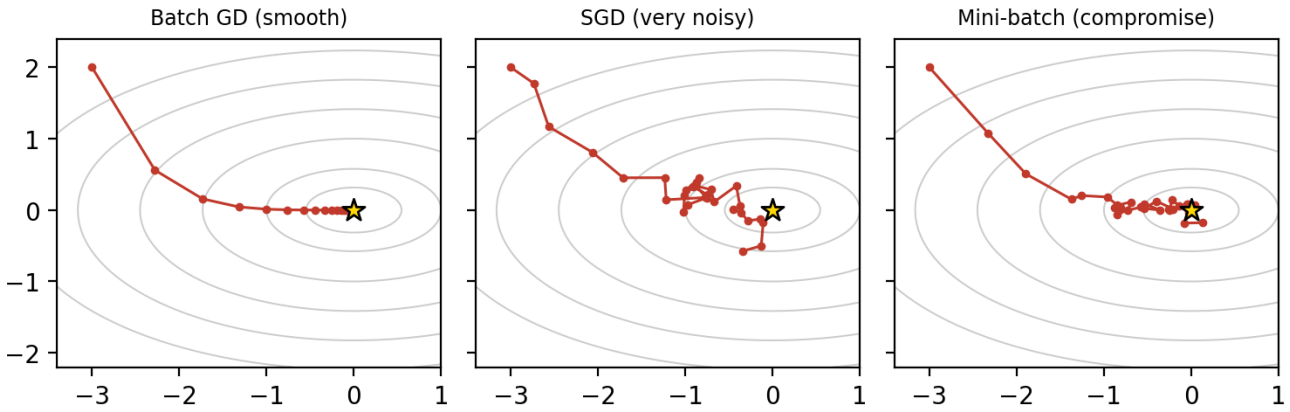


Figure 10: Illustration of the typical paths (SGD and mini-batch use random noise to mimic sampling).

- **Batch GD:** uses the **entire dataset** per update. Stable, smooth path; reaches the global minimum for convex problems. But slow and memory-heavy on big data, with only one update per epoch.
- **Stochastic GD:** updates after **every single sample**. Very fast updates, can escape shallow local minima, works online. But the path is **noisy** and it does not use vectorised hardware well.
- **Mini-batch GD:** uses a **small batch** per update. Balances stability and speed, uses GPU parallelism efficiently, and is the **default in practice** (`model.fit(batch_size=32)`).

Feature	Batch GD	SGD	Mini-batch GD
Data per update	All N samples	1 sample	b samples (e.g. 32)
Updates per epoch	1	N	N / b
Speed per update	Slow	Very fast	Fast
Stability	Very stable	Very noisy	Moderately stable
Memory	High	Low	Medium
Convergence	Smooth	Fluctuates around the minimum	Good compromise
Hardware use	Heavy	Poor	Excellent

Terms: epoch = one full pass through the dataset; **batch size** = samples per update.

$$\text{iterations per epoch} = \frac{N}{\text{batch size}}$$

(Momentum, NAG, AdaGrad, RMSProp and Adam are the *advanced* variants; they are in Module 2.)

13. Quick Revision Sheet

Key formulas

CHEAT SHEET

$$\text{MP}: \quad y = 1 \text{ if } \sum x_i \geq \theta$$

$$\text{Perceptron}: \quad y = \text{step}(\mathbf{w} \cdot \mathbf{x} + b), \qquad w \leftarrow w + \eta(t - y)x$$

$$\text{Sigmoid}: \quad \sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma' = \sigma(1-\sigma) \leq 0.25$$

$$\text{Tanh}: \quad \tanh'(z) = 1 - \tanh^2(z) \qquad \text{ReLU}: \quad \max(0, z)$$

$$\text{GD}: \quad w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

$$\text{MSE} = \frac{1}{N}\sum(y - \hat{y})^2 \qquad \text{BCE} = -[t \log \hat{y} + (1-t)\log(1-\hat{y})]$$

One-line answers

- **Why can't a perceptron solve XOR?** It is a linear classifier and XOR is not linearly separable.
- **Why hidden layers?** To create non-linear decision boundaries and learn feature hierarchies.
- **Why non-linear activations?** Otherwise several layers collapse into one linear layer.
- **Why ReLU over sigmoid?** No saturation for positive z, cheaper, faster training, less vanishing gradient.
- **Why cross-entropy for classification?** Its gradient (ŷ − t) does not vanish when the neuron saturates, and it matches maximum likelihood.
- **What causes vanishing gradients?** Repeated multiplication of derivatives smaller than 1 (such as the sigmoid's 0.25).

Likely exam questions

1. Explain the history of deep learning with a timeline (5 to 10 marks).
2. What is a tensor? Explain ranks with examples. What is a computation graph? Draw one for y = (a + b) × c.
3. Explain the McCulloch-Pitts neuron. Implement AND, OR and NOT with it.
4. Explain the perceptron learning algorithm with an example. State the convergence theorem.
5. Why can't a single-layer perceptron solve XOR? Show how an MLP solves it.
6. State and explain the universal approximation theorem.
7. Compare sigmoid, tanh and ReLU with graphs, formulas, advantages and disadvantages.
8. Compare MSE and cross-entropy. When is each used?
9. What is the vanishing gradient problem? Give its cause, effects and solutions.
10. Differentiate batch, stochastic and mini-batch gradient descent.
11. Short notes: saturation, learning rate, dying ReLU.

Exam-writing tip
Use this flow for every long answer: **Definition → Diagram → Formula → Working / Example → Advantages → Disadvantages → Applications.** Draw neat diagrams: they fetch marks.